```
  ...                                                                    ❺
}
```

❶ Creates the sharding proxy to communicate with Market

❷ Implements the RPC call definition

❸ Retrieves MarketId.marketId from the RPC message to find/create a market actor

❹ Asks Market and transforms the response to an RPC message

❺ Remainder of the RPC call implementations

To communicate with the market, you need access to the sharding regions. Through this proxy, you request the market entity by its ID. If the entity does not exist yet, one is created.

It is important to note that this request differs from the request within entities. There you call `context.ask`, while here you use `[entityRef].ask`. The former uses the context to pass the `ActorRef` to `replyTo` so the callback stays in the context of the actor. Otherwise, the actor's context could be accessed from another thread, breaking the consistency of the actor's state provided by Akka. On the other hand, `market.ask` is not called inside any actor context. `MarketServiceImplSharding` is not an actor. The callback executed upon the response can't break anything here.

The server with this implementation is shown in the next listing.

Listing 12.15 Market server

```
import example.market.grpc.{ MarketServiceHandler, MarketServiceImplSharding }

object MarketServiceServer {

  def init(
      implicit system: ActorSystem[_],
      sharding: ClusterSharding,
      ec: ExecutionContext): Future[Http.ServerBinding] = {
    val marketService: HttpRequest => Future[HttpResponse] =
```